UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA

CARRERA DE INGENIERÍA DE SOFTWARE

ASIGNATURA: INTELIGENCIA DE NEGOCIOS

ENTREGABLE 3: MI PRIMER DASHBOARD

Solución analítica completa con PostgreSQL, Python, Streamlit e Inteligencia Artificial

Dataset analizado:

ESTUDIANTES

Diana Lucía Melena Santander

SANTA ELENA – ECUADOR

2026

DOCUMENTO FINAL DEL PROYECTO

1. Dataset y comprensión del problema

Aspecto	Detalle
Nombre	League of Legends – 2024 Competitive Game dataset
Enlace	https://www.kaggle.com/datasets/barthetur/league-of-legends-2024-competitive-game-dataset
Autor y fuente	barthetur en Kaggle; datos originales de Oracle's Elixir (oracleselixir.com).
Archivo utilizado	`2024_LoL_esports_match_data_from_OraclesElixir.xlsx` (6,3 MB).
Registros y columnas	12 276 filas y 123 columnas, equivalentes a 1 023 partidas (12 filas por partida).
Período	Del 1 de enero al 13 de febrero de 2024 (36 días con partidas).
Cobertura	28 ligas (LCK, LPL, LEC, LCS, CBLOL, PCS, LJL, entre otras), 253 equipos, 151 campeones y 4 parches (13.24 a 14.03).

Corrección respecto al Entregable 1. En la primera entrega se estimaron unas 115 000 filas, 160 columnas y la temporada completa de enero a noviembre. Al descargar y procesar el archivo real se comprobó que la versión publicada en Kaggle corresponde al inicio de la temporada: 12 276 filas, 123 columnas y partidas entre el 1 de enero y el 13 de febrero de 2024. Todo el proyecto se construye sobre estas cifras reales.

Qué representa una fila

Cada fila del archivo original representa la participación de un jugador en una partida. Cada partida genera doce filas: diez de jugadores (cinco por equipo) y dos filas de resumen por equipo, con `position = team`. Mezclar ambos niveles en un mismo cálculo duplicaría cualquier total, por lo que la limpieza los separa.

Variables principales

Tipo	Variables	Uso en el proyecto
Identificadores	gameid, participantid, teamname, playername	gameid + equipo identifica cada registro de hecho.
Temporales	date, split, patch, gamelength	dim_tiempo, dim_liga, dim_parche y la duración.
Categóricas	league, side, position, champion, ban1 a ban5	Dimensiones de liga, lado y campeón; draft.
Numéricas	kills, deaths, assists, totalgold, golddiffat15, dragons, barons, towers, damagetochampions, visionscore	Medidas de la tabla de hechos.
Resultado	result, firstblood, firstdragon, firsttower, datacompleteness	victoria (base de los KPI) y objetivos tempranos.

Problemas de calidad identificados

- **Granularidad mezclada:** filas de jugador y de equipo conviven en la misma tabla.
- **Valores nulos:** 374 registros de equipo (18,3 %) no tienen `golddiffat15`; todos pertenecen a LPL, LDL y DCup, que publican datos parciales.
- **Tipos incorrectos:** `date` llega como texto y `patch` como número, lo que destruye su formato.
- **Categorías vacías:** algunas ligas no informan el split.
- **Columnas sin aporte:** url e identificadores internos.

Justificación, problema y usuarios

Se eligió este dataset porque combina fechas reales, categorías con significado de negocio (liga, equipo, campeón, parche), métricas abundantes y una variable de resultado, lo que permite un modelo dimensional auténtico. El **problema de análisis** es que los equipos y analistas toman decisiones de estrategia y de draft con información dispersa: no existe una vista que relacione rendimiento, ventaja temprana y metajuego por liga y por parche.

Usuario	Uso de la información	Decisión que apoya
Analista o entrenador	Compara su tasa de victoria y su ventaja temprana con los rivales de la liga.	Ajustar la estrategia de los primeros 15 minutos y priorizar objetivos.
Responsable de draft	Revisa presencia, picks, bans y win rate de cada campeón por parche.	Decidir qué campeones banear y cuáles priorizar.
Organizador de la liga	Observa equilibrio entre lados, duración y competitividad.	Detectar desequilibrios que afecten el espectáculo.

2. Preguntas de negocio e indicadores

- **P1.** ¿Qué ventajas de los primeros quince minutos (oro, primer dragón, primera torre) se traducen con mayor frecuencia en victoria?
- **P2.** ¿Qué campeones concentran mayor presencia en el draft y con qué tasa de victoria?
- **P3.** ¿Qué equipos rinden mejor en cada liga y cómo evoluciona su rendimiento?

KPI	Cálculo sobre el modelo	Pregunta	Objeto SQL
Tasa de victoria	SUM(victoria) / COUNT(*) de hecho_partida_equipo, por equipo.	P3	vw_partidas, vw_kpi_equipo, fn_tasa_victoria()
Victorias con ventaja de oro al minuto 15	AVG(victoria) de los registros con oro_dif_15 > 0.	P1	vw_partidas, vw_ventaja_temprana
Presencia en el draft	Partidas en que el campeón fue elegido o baneado / partidas del filtro, sobre hecho_draft.	P2	vw_draft, vw_presencia_campeon, fn_top_campeones()

Ajuste respecto al Entregable 1. El tercer KPI se planteó como el promedio de la diferencia de oro al minuto 15. La validación V5 mostró que ese promedio es exactamente 0 en cualquier liga, porque lo que gana un equipo lo pierde el rival. Por eso el KPI se redefinió como la tasa de victoria de los equipos que llegan al minuto 15 con ventaja de oro, que sí responde a la pregunta P1. El promedio se mantiene como medida en la pestaña Equipo, donde sí es informativo.

3. Limpieza y transformación de datos

La limpieza se implementa en `02_limpieza.py` con pandas y genera tres archivos de staging en `salida/`. La tabla resume cada problema y la acción aplicada:

Problema detectado	Transformación aplicada
Granularidad mezclada: cada partida trae doce filas, diez de jugador y dos de resumen por equipo.	Separación en dos conjuntos mediante la columna position. Las filas de equipo alimentan el hecho principal; las de jugador alimentan el hecho de draft.
La columna date se importa como texto.	Conversión a fecha y hora, y derivación de la columna fecha para enlazar con la dimensión de tiempo.
La columna patch se lee como número y pierde su formato.	Conversión a texto y descomposición en temporada y número de parche.

Problema detectado	Transformación aplicada
Partidas con duración muy baja (remakes por desconexión).	Exclusión de las partidas de menos de quince minutos, por ser valores atípicos que distorsionan los promedios.
Nombres de liga, split y equipo con espacios y variantes.	Normalización de espacios y reemplazo de los split vacíos por la etiqueta Sin split.
Columnas de corte temporal vacías cuando datacompleteness es partial.	Se conserva el nulo de forma deliberada en oro_dif_15, porque distinguir el dato ausente del valor cero es necesario para el KPI.
Columnas de objetivos con nulos en ligas menores.	Relleno con cero, ya que la ausencia de registro equivale a no haber conseguido el objetivo.
Columnas sin aporte analítico (url, identificadores internos).	Descartadas en la selección final de columnas.
Posibles duplicados por partida y equipo.	Eliminación de duplicados sobre la combinación gameid más equipo.

Al ejecutar el script con el archivo real aparecieron tres problemas adicionales que se corrigieron: el dataset se publica en Excel y no en CSV (se cambió a `pd.read_excel`); las columnas enteras con nulos se exportaban como `1729.0` y PostgreSQL las rechazaba (se convierten al tipo entero con nulos `Int64`); y los split vacíos no se reemplazaban porque pandas 3 los conserva como nulos (se usa `fillna("Sin split")`).

```
Windows PowerShell

PS C:\Users\DIANA MAIN\Documents\GitHub\bi-lol-datamart> python 02_limpieza.py
Filas originales: 12,276  |  Columnas: 123
Filas de equipo: 2,046  |  Filas de jugador: 10,230
Remakes descartados (menos de 15 min): 0
Duplicados eliminados: 0
Registros sin oro_dif_15 (datos parciales): 374 (18.3%)

--- RESUMEN ---
Partidas (equipo-partida): 2,046
Partidas unicas:           1,023
Registros de draft:        19,658
Dias en dim_tiempo:        36
Rango de fechas:           2024-01-01 a 2024-02-13
Archivos generados en ./salida/
```

Figura 1. Salida de 02_limpieza.py con los datos reales.

4. Modelo dimensional

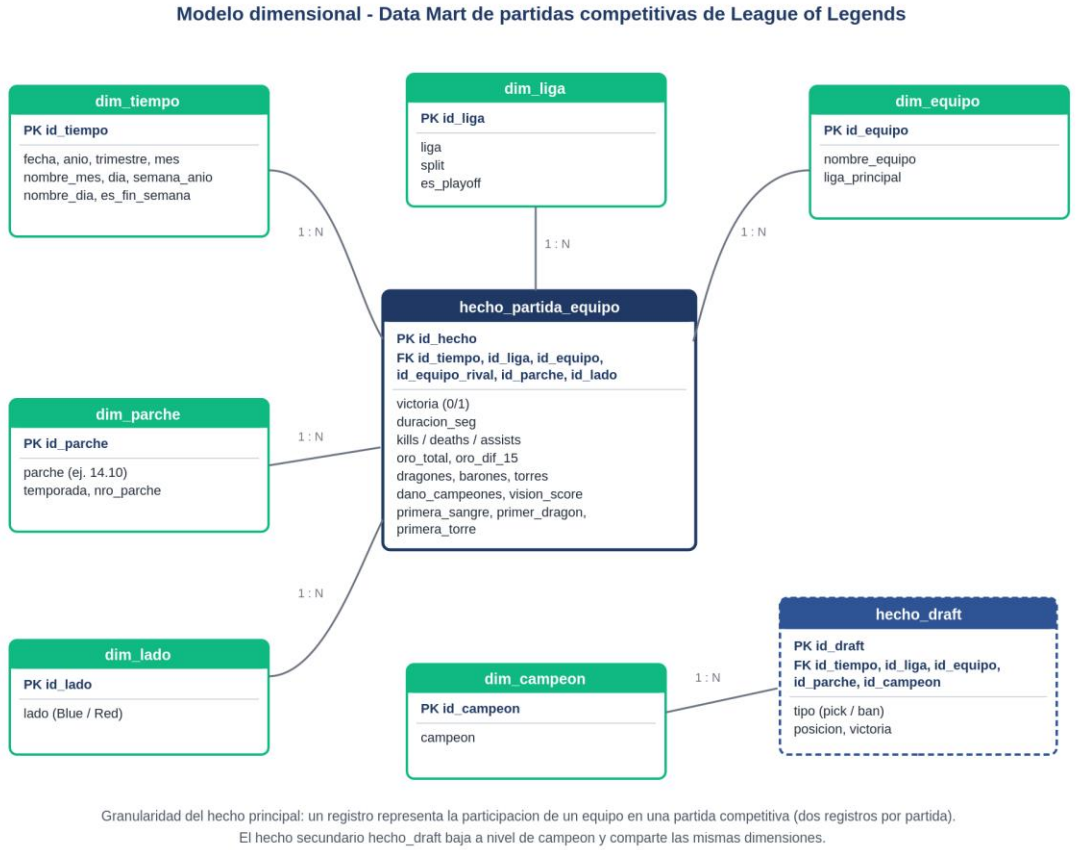


Figura 2. Diagrama estrella del Data Mart.

Granularidad de la tabla de hechos: un registro de hecho_partida_equipo representa la participación de un equipo en una partida competitiva. Cada partida genera, por lo tanto, exactamente dos registros.

Se eligió este nivel y no el de jugador porque los tres KPI definidos en el entregable anterior se calculan sobre resultados de equipo. Para el KPI de presencia en el draft, que sí necesita el detalle de campeón, se incorporó una segunda tabla de hechos, hecho_draft, cuya granularidad es un campeón seleccionado o baneado por un equipo en una partida. Ambas comparten las mismas dimensiones.

Objeto	Tipo	Clave primaria	Observación
`dim_tiempo`	Dimensión	id_tiempo (AAAAMMDD)	Un registro por día del calendario competitivo.
`dim_liga`	Dimensión	id_liga	Combina liga, split y si es fase de playoffs.
`dim_equipo`	Dimensión	id_equipo	Incluye la liga en la que el equipo disputó más partidas.
`dim_parche`	Dimensión	id_parche	Versión del juego; es una dimensión temporal propia del dominio.
`dim_lado`	Dimensión	id_lado	Lado azul o rojo del mapa.
`dim_campeon`	Dimensión	id_campeon	Alimenta el KPI de presencia en el draft.
`hecho_partida_equipo`	Hechos	id_hecho	Seis claves foráneas y quince medidas.
`hecho_draft`	Hechos	id_draft	Cinco claves foráneas; distingue picks de bans.

Medidas asociadas a cada KPI

KPI	Medidas que lo sostienen	Tabla de hechos
Tasa de victoria	victoria (0/1) sobre el conteo de registros, agrupado por equipo.	hecho_partida_equipo
Victorias con ventaja de oro al minuto 15	oro_dif_15 (para seleccionar los registros con ventaja) y victoria. Se complementa con primer_dragon y primera_torre.	hecho_partida_equipo
Presencia en el draft	gameid distintos con el campeón, tipo (pick o ban) y victoria del pick.	hecho_draft

5. Implementación en PostgreSQL

La solución sigue el flujo pedido: dataset de Kaggle, limpieza en Python, carga en PostgreSQL, modelo dimensional, vistas y funciones, y finalmente la aplicación analítica. Los archivos que acompañan a este documento son los siguientes:

Archivo	Contenido
`docker-compose.yml`	Definición del contenedor de PostgreSQL 17 y del volumen que expone la carpeta del proyecto dentro del contenedor.
`01_modelo.sql`	Creación del esquema dm, las seis dimensiones, las dos tablas de hechos, claves primarias, claves foráneas e índices.
`02_limpieza.py`	Lectura del archivo Excel de Kaggle, limpieza, transformaciones y generación de los archivos de carga.
`03_carga.sql`	Tablas de staging, carga con \copy y poblado de dimensiones y hechos.
`04_validaciones.sql`	Consultas de conteo, integridad referencial, nulos y totales.
`05_vistas_funciones.sql`	Cuatro vistas y tres funciones que alimentan el dashboard.
`assets/diagrama_estrella.svg`	Diagrama del modelo dimensional.
`dashboard/ (mockup)`	Mockup navegable del dashboard, separado en index.html, styles.css y app.js.

Configuración con Docker

Se levantó PostgreSQL 17 en un contenedor de Docker en lugar de instalarlo en el sistema operativo. De esta forma el entorno queda aislado y es reproducible en cualquier equipo del grupo, y la base puede iniciarse, detenerse o reconstruirse sin afectar al sistema anfitrión. La definición está en docker-compose.yml:

```
services:
  postgres:
    image: postgres:17
    container_name: bi-postgres
    restart: unless-stopped
    environment:
      POSTGRES_USER: bi_user
      POSTGRES_PASSWORD: bi_password
      POSTGRES_DB: bi_database
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./:/proyecto

volumes:
  postgres_data:
```

Al volumen de datos se añadió un segundo volumen que monta la carpeta del proyecto dentro del contenedor como /proyecto. Sin él, psql no podría leer los scripts ni los archivos CSV generados por la limpieza, porque el contenedor no ve el sistema de archivos del equipo.

Los datos de conexión resultantes son: host localhost, puerto 5433, base bi_database, usuario bi_user y contraseña bi_password. La secuencia completa de ejecución es:

```
docker compose up -d
python 02_limpieza.py
docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/01_modelo.sql
docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/03_carga.sql
docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/04_validaciones.sql
docker compose exec postgres psql -U bi_user -d bi_database -f
/proyecto/05_vistas_funciones.sql

# para apagar el entorno
docker compose down
```

La limpieza se ejecuta en el equipo anfitrión y no dentro del contenedor, porque la imagen de PostgreSQL no incluye Python ni pandas. Sus salidas quedan en ./salida, que el contenedor ve como /proyecto/salida.

Los scripts se ejecutan con psql y no con pgAdmin porque el archivo de carga utiliza el comando \copy, que es una instrucción propia del cliente psql y no una sentencia SQL estándar. Los datos persisten en el volumen postgres_data, de modo que docker compose down no borra la base; para empezar desde cero habría que usar docker compose down -v.

El contenedor publica PostgreSQL en el puerto **5433** del equipo anfitrión. Se cambió desde 5432 porque en el equipo de desarrollo ya existe un PostgreSQL instalado en Windows que ocupa ese puerto: Python se conectaba a esa otra instancia en lugar de la del Data Mart.

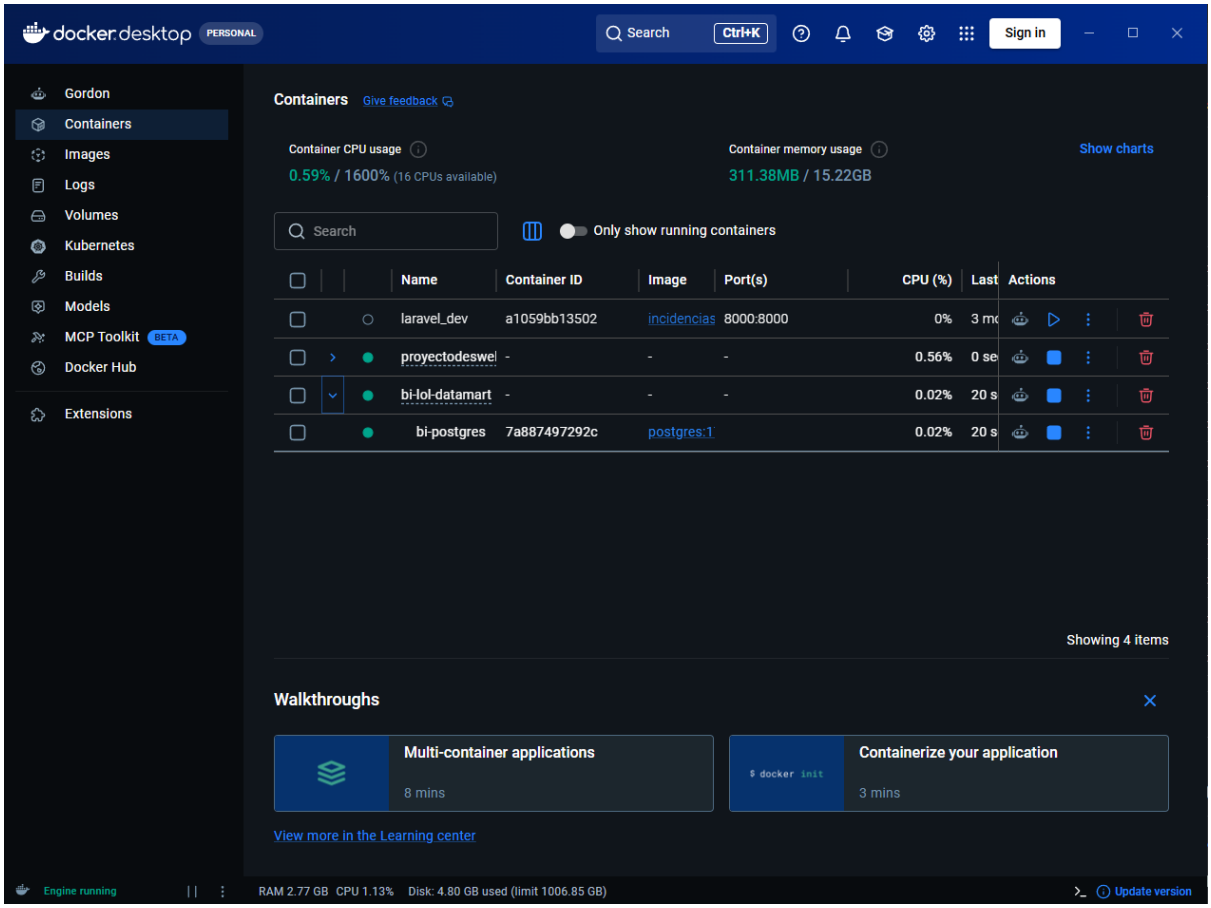


Figura 3. Contenedor bi-postgres en ejecución en Docker Desktop.

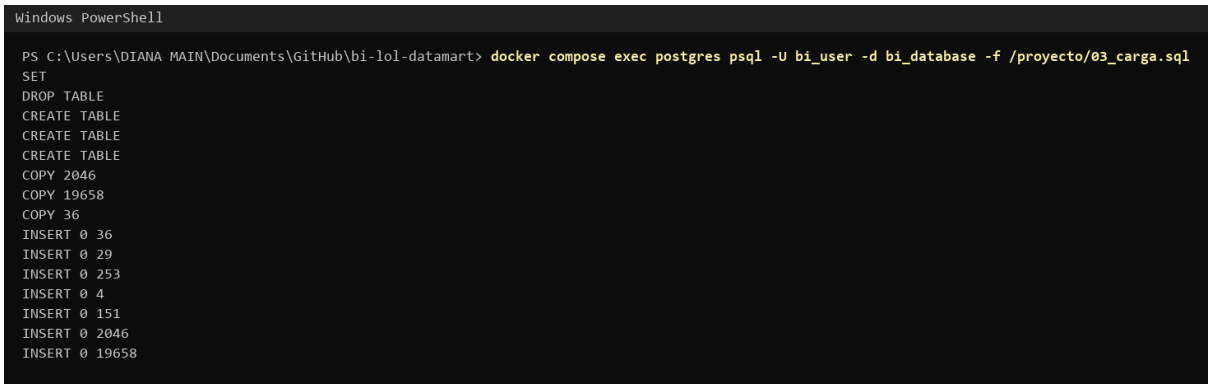


Figura 4. Evidencia de carga: salida de 03_carga.sql.

6. Validación de los datos

El archivo 04_validaciones.sql agrupa cinco bloques. Los resultados esperados son los siguientes:

- V1, conteo por tabla: debe haber dos registros de hecho por cada partida única y todas las dimensiones deben tener filas.
- V2, granularidad: la consulta que busca partidas con un número de equipos distinto de dos debe devolver cero filas, igual que la que busca partidas sin exactamente un ganador.
- V3, integridad referencial: los conteos de hechos huérfanos deben ser cero. También se revisan las dimensiones sin uso, que señalarían filas perdidas durante la carga.
- V4, nulos relevantes: se mide el porcentaje de registros sin oro_dif_15 y se identifica qué ligas los concentran. Este nulo es esperado y documentado.
- V5, totales: la tasa de victoria global debe ser exactamente cincuenta por ciento, porque cada partida tiene un ganador y un perdedor. Si no lo es, hay partidas incompletas en el hecho. También se comprueba la conocida ventaja del lado azul, como validación de sentido del dominio.

La validación V5 es la más útil de todas: un error de carga que duplique o pierda filas se detecta de inmediato, porque la tasa global deja de ser del cincuenta por ciento.

- V6, coherencia con el origen: compara el staging generado desde el archivo original con las tablas de hechos. Partidas, registros de draft, victorias y kills deben coincidir exactamente.

Resultados reales: dos registros de hecho por partida (1 023 partidas, 2 046 filas), cero hechos huérfanos, 18,28 % de nulos en oro_dif_15 concentrados en LPL, LDL y DCup, tasa de victoria global de 50,00 % y coincidencia total entre origen y carga. Además, 12 276 filas originales / 12 filas por partida = 1 023 partidas, igual al conteo del hecho.

```
Windows PowerShell

PS C:\Users\DIANA MAIN\Documents\GitHub\bi-lol-datamart> docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/04_validaciones.sql
SET
        tabla         | registros
----------------------+----------
 dim_campeon          |       151
 dim_equipo           |       253
 dim_lado             |         2
 dim_liga             |        29
 dim_parche           |         4
 dim_tiempo           |        36
 hecho_draft          |     19658
 hecho_partida_equipo |      2046
(8 rows)

 partidas_unicas | filas_hecho | filas_por_partida
-----------------+-------------+-------------------
            1023 |        2046 |              2.00
(1 row)

 gameid | equipos
--------+---------
(0 rows)

 gameid | ganadores
--------+-----------
(0 rows)

 hechos_sin_tiempo
-------------------
                 0
(1 row)

 hechos_sin_equipo
-------------------
                 0
(1 row)

 hechos_sin_liga
-----------------
               0
(1 row)

 nombre_equipo
---------------
(0 rows)
```

Figura 5. Validaciones V1 a V3: conteo, granularidad e integridad referencial.

```
Windows PowerShell

-- (continuacion de la salida de 04_validaciones.sql)

 total | con_oro_dif_15 | sin_oro_dif_15 | pct_nulos | sin_vision_score | sin_rival
-------+----------------+----------------+-----------+------------------+-----------
  2046 |           1672 |            374 |     18.28 |                0 |         0
(1 row)

 liga | partidas | sin_oro_dif_15
------+----------+----------------
 LPL  |      190 |            190
 LDL  |      148 |            148
 DCup |       36 |             36
(3 rows)

 partidas | duracion_media_min | kills_totales | oro_promedio | oro_dif_15_promedio | tasa_victoria_global
----------+--------------------+---------------+--------------+---------------------+----------------------
     1023 |               32.2 |         27993 |        58830 |                   0 |                50.00
(1 row)

 lado | partidas | tasa_victoria
------+----------+---------------
 Red  |     1023 |         45.45
 Blue |     1023 |         54.55
(2 rows)

   desde    |   hasta    | dias
------------+------------+------
 2024-01-01 | 2024-02-13 |   36
(1 row)
```

Figura 6. Validaciones V4 y V5: nulos relevantes y totales.

```
Windows PowerShell

 -- V6. Coherencia entre datos originales (staging) y datos cargados

           control           | origen | cargado
-----------------------------+--------+---------
 partidas (equipo-partida)   |   2046 |    2046
 partidas unicas             |   1023 |    1023
 registros de draft          |  19658 |   19658
 victorias                   |   1023 |    1023
 kills totales               |  27993 |   27993
(5 rows)
```

Figura 7. Validación V6: coherencia entre datos originales y cargados.

```
Windows PowerShell

PS C:\Users\DIANA MAIN\Documents\GitHub\bi-lol-datamart> docker compose exec postgres psql -U bi_user -d bi_database
SET
-- Vista vw_kpi_equipo (LCK, top 6 por tasa de victoria)
       equipo        | liga | split  | partidas | victorias | tasa_victoria | oro_dif_15_prom | duracion_media_min | dragones_prom | barones_prom
---------------------+------+--------+----------+-----------+---------------+-----------------+--------------------+---------------+--------------
 Gen.G               | LCK  | Spring |       14 |        12 |         85.71 |            1362 |               31.1 |          3.21 |         0.93
 T1                  | LCK  | Spring |       14 |        11 |         78.57 |            2187 |               29.6 |          2.36 |         1.00
 Hanwha Life Esports | LCK  | Spring |       13 |        10 |         76.92 |             416 |               30.3 |          2.46 |         0.69
 KT Rolster          | LCK  | Spring |       16 |        10 |         62.50 |             178 |               33.1 |          2.63 |         0.69
 Kwangdong Freecs    | LCK  | Spring |       13 |         6 |         46.15 |            -609 |               33.3 |          2.62 |         0.85
 FearX               | LCK  | Spring |       14 |         6 |         42.86 |            -540 |               31.2 |          2.07 |         0.71
(6 rows)

-- Funcion fn_tasa_victoria('LCK', '2024-01-01', '2024-02-13')
       equipo        | partidas | victorias | tasa_victoria
---------------------+----------+-----------+---------------
 Gen.G               |       14 |        12 |         85.71
 T1                  |       14 |        11 |         78.57
 Hanwha Life Esports |       13 |        10 |         76.92
 KT Rolster          |       16 |        10 |         62.50
 Kwangdong Freecs    |       13 |         6 |         46.15
 FearX               |       14 |         6 |         42.86
(6 rows)

-- Funcion fn_top_campeones('14.02', 6)
 campeon | presencia_pct | picks | bans | win_rate
---------+---------------+-------+------+----------
 Karma   |         85.86 |    31 |  230 |    54.84
 Varus   |         84.54 |   152 |  105 |    48.03
 K'Sante |         79.28 |   164 |   77 |    56.71
 Ashe    |         78.95 |    49 |  191 |    59.18
 Kalista |         78.62 |    88 |  151 |    53.41
 Azir    |         73.68 |   136 |   88 |    52.21
(6 rows)

-- Funcion fn_evolucion_equipo('T1')
 anio | mes | nombre_mes | partidas | tasa_victoria | oro_dif_15_prom
------+-----+------------+----------+---------------+-----------------
 2024 |   1 | enero      |       10 |         70.00 |            1813
 2024 |   2 | febrero    |        4 |        100.00 |            3122
(2 rows)
```

Figura 8. Vistas y funciones SQL consultadas con los datos cargados.

7. Mockup de la aplicación

La aplicación se organiza en cuatro vistas. Vista general reúne los tres KPI, los dos gráficos y la tabla de detalle. Equipos profundiza en el rendimiento individual de cada equipo con su evolución mensual. Metajuego concentra el análisis de campeones por parche. Detalle de partidas permite llegar al registro individual para auditar cualquier número mostrado arriba. Los filtros de liga, split, fechas, parche y equipo son comunes a todas las vistas.

Corrección incorporada: el mockup proponía cuatro vistas y la aplicación final mantiene esa organización (Vista general, Metajuego, Equipo y Detalle de partidas). Se agregó el filtro de mínimo de partidas, porque un equipo con 2 partidas y 100 % de victorias distorsiona el ranking.

Mockup navegable: https://misstery13.github.io/bi-lol-datamart/dashboard/

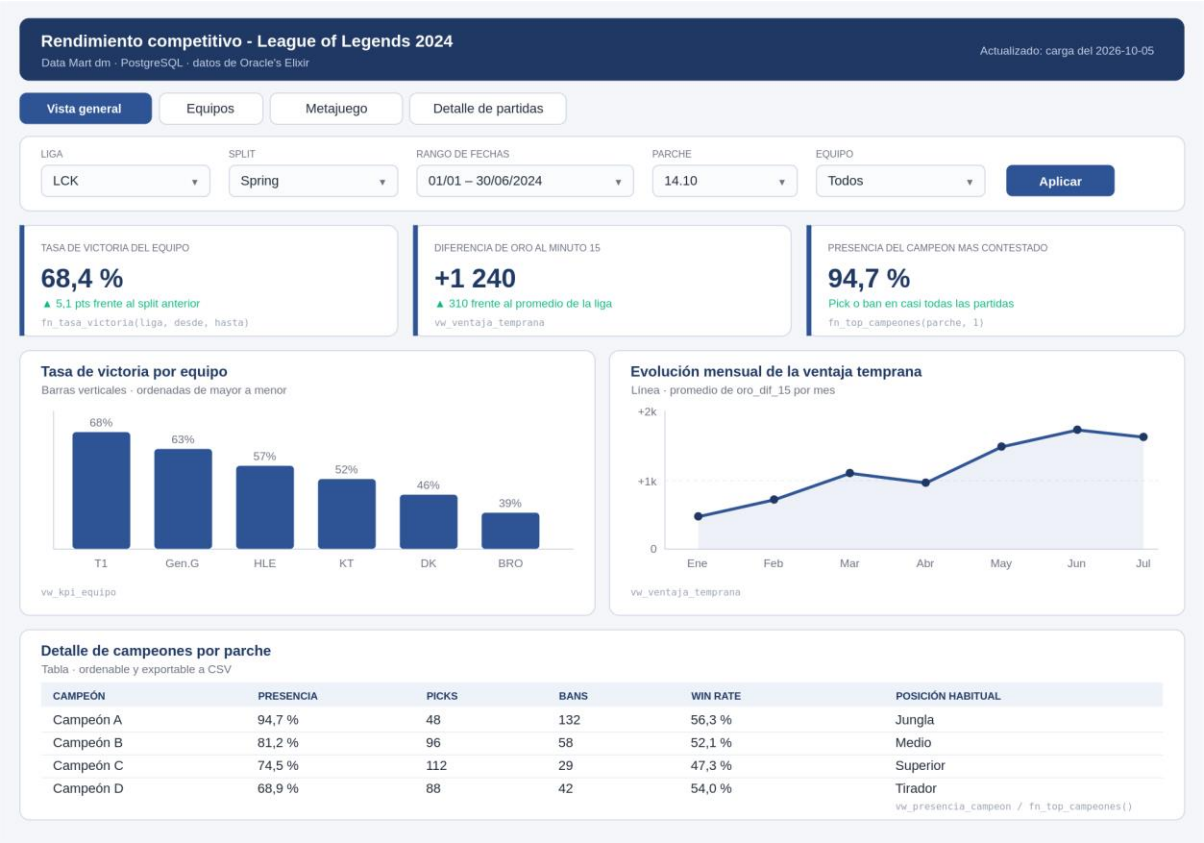


Figura 9. Mockup de la vista general del dashboard.

8. Matriz de trazabilidad

Componente	Pregunta / KPI	Filtros	Datos utilizados	Vista / Función SQL
Tarjeta KPI 1	Mejor tasa de victoria (P3)	Liga, split, fechas, parche, mínimo de partidas	hecho_partida_equipo + dim_equipo, dim_liga, dim_tiempo, dim_parche	vw_partidas
Tarjeta KPI 2	Victorias con ventaja de oro al min 15 (P1)	Liga, split, fechas, parche	hecho_partida_equipo (oro_dif_15, victoria)	vw_partidas
Tarjeta KPI 3	Presencia del campeón más contestado (P2)	Liga, split, fechas, parche	hecho_draft + dim_campeon, dim_parche	vw_draft
Gráfico de barras	¿Qué equipos rinden mejor? (P3)	Liga, split, fechas, parche, mínimo de partidas	hecho_partida_equipo + dim_equipo	vw_partidas
Gráfico de líneas	¿Cuánto pesa la ventaja temprana? (P1)	Liga, split, fechas, parche	hecho_partida_equipo + dim_tiempo	vw_partidas
Barras de picks y bans	¿Qué campeones dominan? (P2)	Liga, split, fechas, parche	hecho_draft + dim_campeon	vw_draft
Tabla de metajuego	Presencia, picks, bans, win rate (P2)	Liga, split, fechas, parche	hecho_draft + dim_campeon	vw_draft
Pestaña Equipo	Evolución de un equipo (P3)	Equipo + filtros globales	hecho_partida_equipo, hecho_draft	fn_evolucion_equipo (), vw_draft
Detalle de partidas	Auditoría de cualquier cifra	Liga, split, fechas, parche	hecho_partida_equipo + todas las dimensiones	vw_partidas
Filtros globales	Acotan todos los componentes	—	dim_liga, dim_tiempo, dim_parche	SELECT DISTINCT sobre cada dimensión

9. Aplicación Python + Streamlit

La aplicación está en `app/app.py`. Se conecta a PostgreSQL con `psycopg2` y obtiene **todos** los datos de las vistas y funciones del esquema `dm`; no lee el archivo original ni los CSV de staging.

```
@st.cache_resource
def conexion():
    return psycopg2.connect(host=os.getenv("PGHOST", "localhost"),
                            port=os.getenv("PGPORT", "5433"),
                            dbname="bi_database", user="bi_user", ...,
                            options="-c search_path=dm")
```

Los filtros de la barra lateral (liga, split, rango de fechas, parche y mínimo de partidas) se envían como **parámetros** a una misma cláusula WHERE que usan todas las consultas. Una lista vacía significa "sin filtro". Los valores nunca se concatenan al texto SQL, lo que evita la inyección de SQL:

```
WHERE (%(ligas)s::text[]   IS NULL OR liga   = ANY(%(ligas)s))
  AND (%(splits)s::text[]  IS NULL OR split  = ANY(%(splits)s))
  AND (%(parches)s::text[] IS NULL OR parche = ANY(%(parches)s))
  AND fecha BETWEEN %(desde)s AND %(hasta)s
```

Pestaña	Contenido
Vista general	Tres KPI, gráfico de barras de tasa de victoria por equipo y gráfico de líneas semanal de victorias según la ventaja temprana.
Metajuego	Gráfico de barras apiladas de picks y bans y tabla de presencia, win rate y posición habitual de cada campeón.
Equipo	Indicadores de un equipo, su evolución mensual (fn_evolucion_equipo) y sus campeones más usados.
Detalle de partidas	Tabla de registros de hecho filtrados, descargable en CSV.

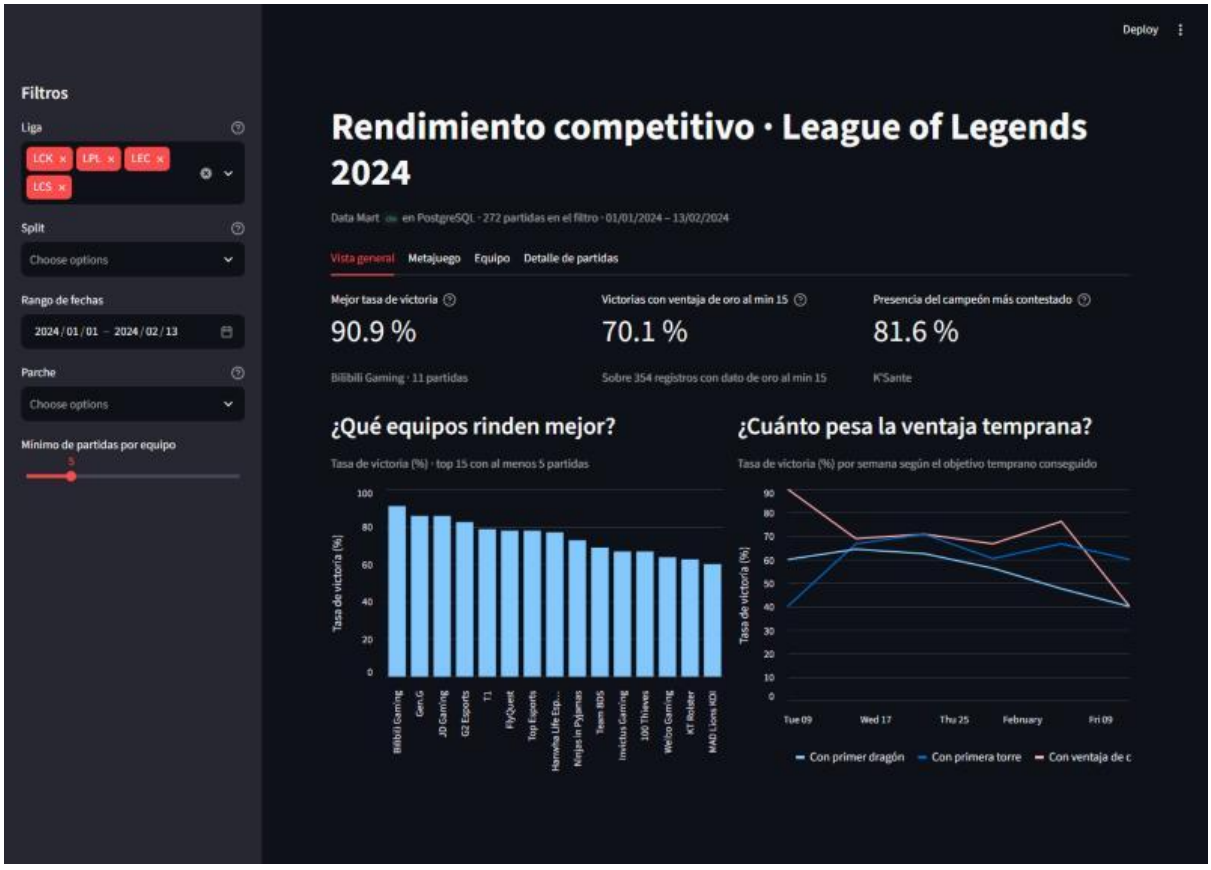


Figura 10. Aplicación Streamlit: vista general con los filtros por defecto.

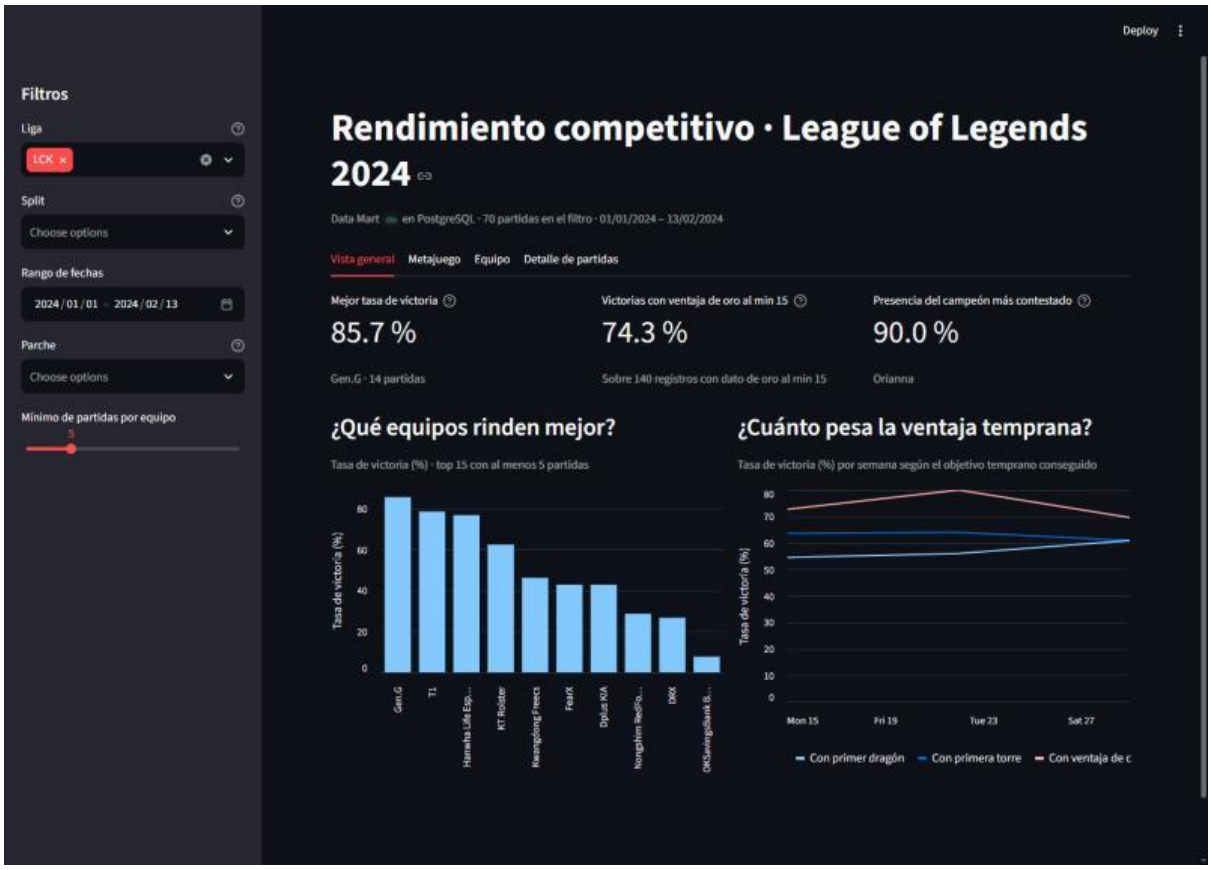


Figura 11. Los mismos componentes después de filtrar solo la liga LCK.



Figura 12. Pestaña Metajuego.

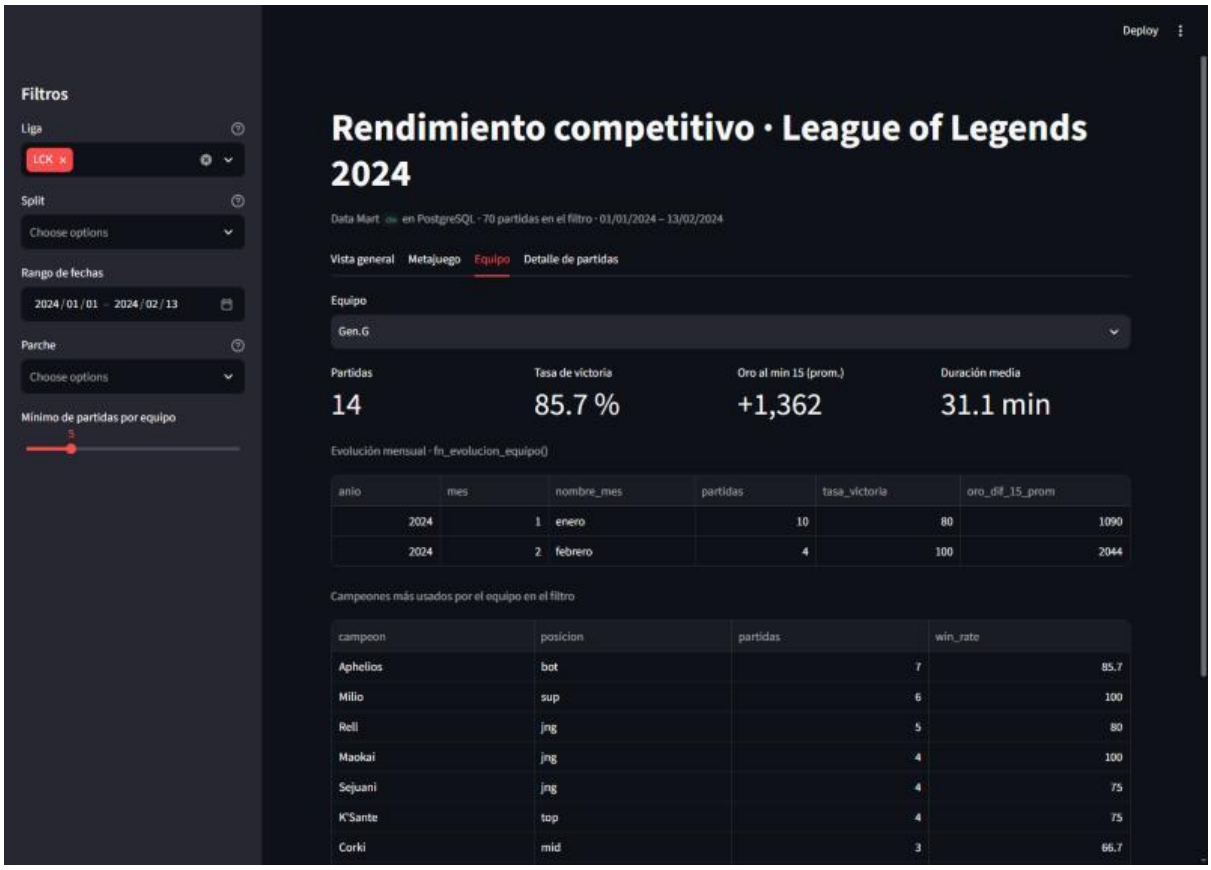


Figura 13. Pestaña Equipo.

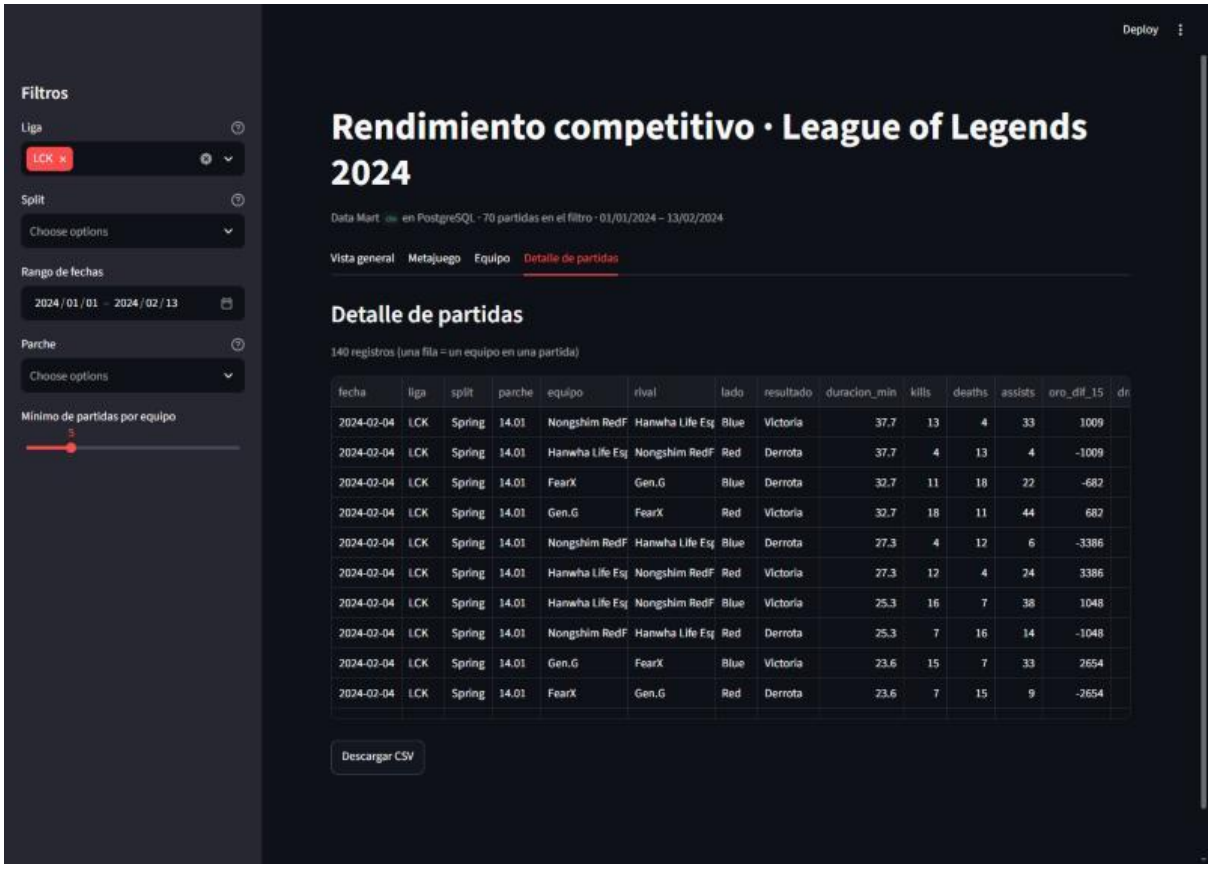


Figura 14. Pestaña Detalle de partidas.

Resultado relevante para la toma de decisiones

Con las cuatro ligas principales seleccionadas (LCK, LPL, LEC y LCS), los equipos que llegan al minuto 15 con ventaja de oro ganan el **70,1 %** de las partidas. Al filtrar solo la LCK la cifra sube a **74,3 %**: en esa liga una ventaja temprana se convierte en victoria con más frecuencia que en el resto. En el mismo filtro, Gen.G lidera con 85,7 % de victorias y Orianna está presente en el 90 % de las partidas, elegida o baneada.

Decisión que apoya: un equipo que enfrenta a rivales de la LCK debería preparar su plan de juego para no ceder ventaja en los primeros 15 minutos (control de la jungla, primer dragón y primera torre) y decidir en el draft si banea a Orianna o la prioriza, porque casi ningún rival la deja libre.

10. Uso de Inteligencia Artificial

Se utilizó Claude Code (Anthropic) como asistente durante el desarrollo. Se documentan cuatro usos significativos:

Uso 1. Error al cargar los datos en PostgreSQL

Aspecto	Detalle
Problema	Al ejecutar 03_carga.sql, \copy fallaba con: invalid input syntax for type integer: "1729.0".
Prompt o consulta	Se le pidió ejecutar todo el proceso con el archivo real y corregir los errores que aparecieran.
Solución propuesta	Identificó que pandas convierte a decimal las columnas enteras que tienen nulos (oro_dif_15) y propuso convertirlas al tipo Int64, que admite nulos sin decimales.
Qué se utilizó o modificó	Se agregó en 02_limpieza.py la conversión a Int64 de las columnas numéricas antes de exportar.
Cómo se comprobó	Se volvió a generar el staging y la carga terminó sin errores. Las validaciones V1 y V6 confirman 2 046 registros cargados, iguales a los del origen.

Uso 2. La aplicación no se conectaba a la base de datos

Aspecto	Detalle
Problema	Streamlit mostraba UnicodeDecodeError: 'utf-8' codec can't decode byte 0xf3 al conectar con psycopg2.
Prompt o consulta	Se le pidió levantar la aplicación y resolver el error de conexión.
Solución propuesta	Explicó que el byte 0xf3 es una "ó" de un mensaje de error en español, y que el PostgreSQL de Docker responde en inglés. Revisó qué proceso ocupaba el puerto 5432 y encontró un PostgreSQL instalado en Windows.
Qué se utilizó o modificó	Se cambió el puerto publicado por el contenedor a 5433 en docker-compose.yml y en la conexión de la aplicación, sin tocar el servicio de Windows.
Cómo se comprobó	Una consulta de prueba desde Python a localhost:5433 devolvió 2 046 hechos, igual que dentro del contenedor.

Uso 3. Los gráficos de Streamlit no se mostraban

Aspecto	Detalle
Problema	Error: pyarrow requires NumPy 2.0 or newer, found 1.26.4.
Prompt o consulta	Mismo pedido: dejar la aplicación funcionando.
Solución propuesta	Propuso dos opciones: actualizar NumPy o instalar una versión anterior de pyarrow. Recomendó la segunda porque otras librerías del equipo dependen de NumPy 1.26.
Qué se utilizó o modificó	Se instaló pyarrow 21 y se dejó fijada la versión en requirements.txt.
Cómo se comprobó	Los gráficos se muestran y las demás librerías siguen funcionando.

Uso 4. Filtros que afecten realmente a las consultas

Aspecto	Detalle
Problema	El enunciado exige que los filtros cambien las consultas y no solo la tabla mostrada.
Prompt o consulta	Se compartió el enunciado del Entregable 3 y se pidió construir la aplicación sobre las vistas del Data Mart.
Solución propuesta	Propuso una sola cláusula WHERE parametrizada en la que una lista vacía equivale a "sin filtro" (IS NULL OR columna = ANY(lista)), y una vista nueva vw_draft para poder filtrar el metajuego por liga y fecha.
Qué se utilizó o modificó	Se creó vw_draft en 05_vistas_funciones.sql y la cláusula se reutiliza en todas las consultas de app.py.
Cómo se comprobó	Al filtrar solo LCK, los KPI, gráficos y tablas cambian (Figuras de la sección 9) y los números coinciden con fn_tasa_victoria('LCK', ...) ejecutada directamente en psql.

11. Ejecución y entrega

Repositorio: https://github.com/Misstery13/bi-lol-datamart

Dataset: https://www.kaggle.com/datasets/barthetur/league-of-legends-2024-competitive-game-dataset

Video de presentación:
https://drive.google.com/file/d/1rPyXhh2Nnhdeo60MUhU4wRjrCm_YOu63/view?usp=sharing

```
pip install -r requirements.txt
docker compose up -d
python 02_limpieza.py
docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/01_modelo.sql
docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/03_carga.sql
docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/04_validaciones.sql
docker compose exec postgres psql -U bi_user -d bi_database -f /proyecto/05_vistas_funciones.sql
streamlit run app/app.py
```

El README del repositorio detalla cada paso. El video de presentación muestra el proyecto funcionando: modelo dimensional, consultas en PostgreSQL y la aplicación Streamlit con sus filtros.

12. Conclusión

¿Cómo se almacena la información? En un Data Mart en PostgreSQL con esquema estrella: seis dimensiones y dos tablas de hechos con claves primarias, foráneas e integridad verificada.

¿Cómo se consulta? Mediante vistas que aplanan los hechos con sus dimensiones y funciones parametrizadas, consumidas desde Python con consultas parametrizadas.

¿Cómo se presenta? En una aplicación Streamlit de cuatro pestañas cuyos filtros recalculan los KPI, gráficos y tablas directamente en la base de datos.

¿Qué decisiones apoya? Permite a analistas y entrenadores comparar el rendimiento por liga y equipo, medir cuánto pesa la ventaja temprana y preparar el draft con la presencia real de cada campeón. Además, la validación muestra que el lado azul gana el 54,55 % de las partidas, un dato útil para un organizador de liga.